
LOCAL-ENGINE-INTERNET

Internals of this repo

A file-by-file account of the FastAPI process on :8090. Search, fetch, and an OpenAI-compatible tool loop in front of `UPSTREAM_LLM_BASE_URL`. After this document you should be able to open any file under `engine/` and already know what it does.

Package: `engine` · version 0.1.0 · Python $\geq$ 3.12

Toggle: <http://127.0.0.1:8090/ui>

OpenAPI: <http://127.0.0.1:8090/docs>

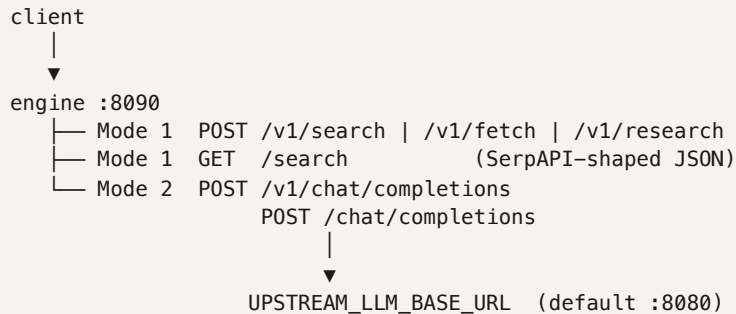
Source of truth for this paper: the Python in this repository, not comments elsewhere.

Contents

1. What the process does
2. Process start – main, runtime, config, auth
3. Plugin – two layers
4. HTTP surface – every route and status
5. Mode 1 – search, fetch, research, GET /search
6. Mode 2 – chat, loop, tools, DSML
7. Search mix – providers, merge, query
8. Fetch ladder – policy, tiers, PDF, SEC, JS
9. Supporting modules
10. File map of the repo
11. Tests – what each file locks
12. Scripts and Docker
13. Environment
14. How to run

1. What the process does

One FastAPI app (`engine.main:app`). It does not load weights. It talks to search APIs, downloads pages, and — only in chat — sits between the client and an upstream OpenAI-compatible LLM (default `http://127.0.0.1:8080`, typically ai-router).



Mode 1 — the caller already decided to search or fetch. No LLM in the hop. Plugin off → HTTP 503.

Mode 2 — the caller sends chat messages. The engine may inject tools, run them, and call the LLM again until it answers in prose. Plugin off → pass-through (no tools).

Same search code (`engine/search/merge.py`) and fetch code (`engine/fetch/ladder.py`) under both modes. Fail-closed: tool and search errors are `ERROR: ... Do not present unverified facts as sourced`. The string “answer from your own knowledge” is never emitted.

Two modes are never mixed in one hop. A Mode 1 request never calls the Brain. A Mode 2 request only searches if the tool loop runs a tool.

2. Process start – main, runtime, config, auth

python -m engine -- engine/__main__.py

CLI: run (default), setup, check, or help. Unknown non-flag first args exit 2. run prints the connect banner then starts uvicorn on engine.main:app with reload=False. setup delegates to engine/setup.py. check runs scripts/check_search.py via runpy.

Console script alias from pyproject.toml: local-engine → engine.__main__:main.

engine/main.py

Creates the FastAPI app (title “Phoenician local-engine-internet”, version 0.1.0) with a lifespan:

1. Build a shared `httpx.AsyncClient`: `User-Agent local-engine-internet/0.1, follow_redirects=True, connect/write timeout = DOC_FETCH_TIMEOUT (20s), read timeout = REQUEST_TIMEOUT (300s)`.
2. Build `FetchCache` with `FETCH_CACHE_TTL (1800s)` and `FETCH_CACHE_FAILURE_TTL (120s)`. Stored on `runtime.fetch_cache`. The ladder also keeps its own process cache (`_PROCESS_CACHE`); Mode 1/2 fetch currently uses that module-level cache unless a caller passes one in.
3. Copy `settings.plugin_enabled` into `runtime.plugin_enabled`.
4. Set `runtime.capabilities`: `brain_tool_calls_supported=False`, a canary note, and providers from `configured_providers()`.
5. If no providers: log a warning. Search will 502 until a key is set.
6. If `RUN_CANARY_ON_STARTUP`: schedule `run_canary` as a background task. Listen is not blocked if the Brain is down. Timeout is `CANARY_TIMEOUT_SECONDS (5s)`.
7. On shutdown: cancel the canary, close the `httpx` client.

CORS: origins from `CORS_ALLOWED_ORIGINS` (comma-separated). If that list is empty *and* `ENGINE_API_KEY` is unset, origins become `*` so a local browser UI can attach. Methods GET/POST. Headers: Authorization, Content-Type, X-Priority, X-Phoenician-Plugin. `allow_credentials=False`.

Routers, in mount order:

Router	Auth	File
health	none	api/health.py
ui	none	api/ui.py
openai discovery	none	api/openai_compat.py
plugin	require_engine_key	api/plugin.py
chat	same	api/chat.py
primitives	same	api/primitives.py
compat	same	api/compat.py

When the key is unset, `require_engine_key` is a no-op, so plugin/chat/search are open. When the key is set, /ui still loads but its `fetch("/v1/plugin")` returns 401 unless the browser sends Bearer — the page explains that.

engine/runtime.py

Process-wide handles, not a class:

- `http_client` — the shared `httpx` client. `get_http_client()` raises if lifespan has not run.

- `fetch_cache` — created in lifespan; `get_fetch_cache()` same guard. (The ladder's own `_PROCESS_CACHE` is what `fetch_url` uses by default.)
- `plugin_enabled` — Layer 1 boolean. Flipped by `POST /v1/plugin` or set from env at start.
- `capabilities` — filled by the canary: `brain_tool_calls_supported`, `canary_error`, `canary_model`, `providers`.

engine/config.py

Reads `.env` once via `load_dotenv()` at import. Tests mutate the settings singleton in place; they do not reload the environment.

Enums:

- `FetchMode`: `full`, `no_browser`, `httpx_only`. Unknown string → `no_browser`.
- `DomainMode`: `general_research`, `ir_discovery`, `earnings_doc`. Unknown → `general_research`.
- `WebPolicy`: `auto`, `required`, `off`. Unknown → `auto`.

Helpers `_int_env` / `_float_env` / `_bool_env`: empty string keeps the default. Bool true values: `1`, `true`, `yes`, `on`.

Aliases: `SEARCH_API_KEY` → `ENGINE_API_KEY`. `SERPAPI_API_KEY` → `SERPAPI_KEY`. `ROUTER_API_KEY` → `UPSTREAM_API_KEY`.

`VLLM_REQUIRED_FLAGS` (reported on `/capabilities`, logged if the canary fails):

```
--tokenizer-mode deepseek_v4
--tool-call-parser deepseek_v4
--enable-auto-tool-choice
--reasoning-parser deepseek_v4
```

engine/auth.py

If `ENGINE_API_KEY` is empty, return immediately. Else accept Authorization: Bearer <key> or query ?
api_key= (so SerpAPI-shaped GET `/search` can drop in). Anything else → 401 Missing or invalid engine API key.

Startup canary – run_canary in agent/loop.py

One forced `web_search` tool-choice against the upstream model: “Call `web_search` with query 'phoenician capital'.” `max_tokens=256`, `stream=False`. Success = structured `tool_calls` or DSML recovery from content. Failure is stored; it does not disable Mode 1. Mode 2 still runs — a Brain that cannot emit tools will just answer in prose (or 424 if policy is required).

3. Plugin – two layers

Code: `engine/plugin.py`. HTTP: `engine/api/plugin.py`, `engine/api/ui.py`.

Layer 1 – enabled or disabled

`plugin_enabled(web, headers)`:

1. If `runtime.plugin_enabled` is `False` → `False`. A request cannot force the plugin on. Header `on` / body `enabled: true` are ignored when the process switch is off.
2. Header `X-Phoenician-Plugin` (any casing): `off` / `0` / `false` / `disabled` → `False`. `on` / `1` / `true` / `enabled` is parsed but cannot override a global off.
3. Body `phoenician_web.enabled`: string in `1/true/yes/on` or a boolean. Only `false` changes anything — it forces this request off.
4. Otherwise `True`.

When Layer 1 is off:

- Mode 1: `require_plugin_enabled` raises 503 with detail `Internet plugin is disabled`. Enable it at `/ui` or `POST /v1/plugin {"enabled": true}`.
- Mode 2: pass-through. `phoenician_*` stripped. This engine's tools stripped from `tools`. If that empties the list, `tools` is removed; `tool_choice` is removed when it is `auto`, `required`, or names one of our three functions. No system guidance injected.

Set Layer 1 at `/ui`, `POST /v1/plugin {"enabled": true|false}` (writes `runtime.plugin_enabled`), or `PLUGIN_ENABLED` at startup.

Layer 2 – intelligence (only if Layer 1 is on)

`intelligence_policy`: if Layer 1 is off, returns `WebPolicy.OFF` regardless of body. Else reads `phoenician_web.policy` or `DEFAULT_WEB_POLICY`.

Value	Behaviour
<code>auto</code>	Tools injected. The model decides whether to call them.
<code>required</code>	Must complete a successful <code>web_search</code> (or <code>search_and_read</code> , which calls <code>web_search</code>). One forced re-ask if the first turn has no tool call. Still none, or every provider failed → HTTP 424.
<code>off</code>	Pass-through this request. Required for <code>stream: true</code> .

`phoenician_tools`: `[]` is also pass-through (managed tool list empty). Omitted `phoenician_tools` means all three tools. Unknown names in that list are dropped.

`plugin_status()`

Returns `enabled`, `intelligence` (or `off` when Layer 1 is off), `global_enabled`, `layers` (`1_master`, `2_intelligence`), and a short explain dict. GET `/v1/plugin` adds `providers` and `search_ready` (at least one configured provider *and* plugin on).

`engine/api/ui.py`

GET `/ui` returns an HTML page. JS GET/POST `/v1/plugin`. 401 and network errors are shown as text, not a silent blank. Port in the snippet is substituted from `settings.port`.

4. HTTP surface – every route and status

Unauthenticated (always)

Method / path	File	Returns
GET /	health.py	Welcome: <code>search_ready</code> , plugin, providers, connect snippets, curl examples, hint if no keys.
GET /health	health.py	<code>status: ok</code> , plugin, upstream URL, providers, <code>fetch_mode</code> .
GET /metrics	health.py	Prometheus text (<code>CONTENT_TYPE_LATEST</code>).
GET /capabilities	health.py	Canary result, per-provider bools, vLLM flags, tools list.
GET /ui	ui.py	HTML toggle.
GET /v1	openai_compat.py	Discovery: endpoints, snippets, notes.
GET /v1/models, /models	openai_compat.py	One model card: <code>settings.upstream_model</code> .
GET /v1/models/{id}	openai_compat.py	That card, or 404 Unknown model.

Authenticated when `ENGINE_API_KEY` is set

Method / path	File	Success body (short)
GET /v1/plugin	plugin.py	status + providers + <code>search_ready</code>
POST /v1/plugin	plugin.py	same after writing <code>runtime.plugin_enabled</code>
POST /v1/search	primitives.py	hits, providers_used, errors, confirmed
POST /v1/fetch	primitives.py	Page dict (ok may be false – still HTTP 200)
POST /v1/research	primitives.py	hits + pages (confirmed URLs fetched first)
GET /search	compat.py	SerpAPI-shaped JSON; pins SerpAPI when configured
POST /v1/chat/completions	chat.py	upstream JSON + <code>phoenician_*</code> extras
POST /chat/completions	chat.py	same handler

Status codes this process emits

Code	When	Typical detail
400	Chat body is not JSON	Request body must be JSON
400	Chat <code>stream:true</code> and the loop would run	<code>stream:true</code> and the internet plugin cannot be combined yet – disable the plugin or set <code>phoenician_web.policy</code> to off.
400	Pydantic: empty <code>query</code> on Mode 1	validation error
401	Key set, missing/wrong Bearer or <code>api_key</code>	Missing or invalid engine API key
404	Unknown model id	Unknown model
424	<code>policy=required</code> and no successful search	model did not perform a successful <code>web_search</code> or <code>web_search</code> was required but every provider failed
502	Every search provider failed / none configured / pinned providers missing	the <code>ERROR: web_search unavailable (...)</code> string
502	Upstream LLM HTTP ≥ 400 or connect error	Upstream LLM failed: HTTP ... or Upstream LLM unreachable (...). Search still works at <code>POST /v1/search</code> . (If plugin off, that sentence says <code>enable /ui</code> instead.)
503	Mode 1 while plugin off	enable at <code>/ui</code> or <code>POST /v1/plugin</code>

Failed `POST /v1/fetch` is still 200 with `ok: false` and error. Tool failures inside Mode 2 are strings in `role: tool`, not HTTP errors (except 424 for required search).

Prometheus – `engine/metrics.py`

Name	Type	Labels
<code>engine_requests_total</code>	Counter	endpoint, status – chat uses success/error/424; search uses success/502; fetch uses success/error; research uses success
<code>engine_active_requests</code>	Gauge	chat in-flight only
<code>engine_request_latency_seconds</code>	Histogram	endpoint=chat
<code>engine_search_total</code>	Counter	provider, status=ok error
<code>engine_fetch_total</code>	Counter	tier (httpx, curl_cffi, playwright, sec, cache, ...), status=ok error
<code>engine_tool_rounds_total</code>	Counter	incremented once per completed tool round in the loop

5. Mode 1 – search, fetch, research, GET /search

Code: engine/api/primitives.py, engine/api/compat.py. Every handler calls `require_plugin_enabled` first.

POST /v1/search

Body (SearchRequest):

```
{
  "query": "Phoenician Capital",    // required, stripped; empty → 400
  "num_results": 8,                // 1..20
  "providers": null,               // optional pin list
  "domain_mode": "general_research",
  "recency_days": null,
  "on_empty": "empty"             // "empty" | "error"
}
```

Calls `run_search`. `SearchUnavailable` → 502. Success:

```
{
  "query": "...",
  "hits": [ { title, url, snippet, source, kind, position, date, query, also_from } ],
  "providers_used": ["serpapi", "tavily"],
  "errors": ["brave: failed (...)],
  "confirmed": ["https://..."]      // organics with also_from
}
```

`on_empty=error` and zero hits after merge → 502. Default empty returns hits: `[]`.

POST /v1/fetch

```
{"url": "https://example.com", "max_chars": 8000, "mode": "httpx_only", "domain_mode":
"general_research"}
```

`mode` maps to `FetchMode` (same parser as `env`). Returns the `Page` dataclass as a dict: `url`, `final_url`, `title`, `text`, `content_type`, `is_pdf`, `tier`, `ok`, `error`.

POST /v1/research

Search, then fetch. `ResearchRequest`: `query` (required), `fetch_top` 1..8 (default 3), `max_chars_per_page` 200..50000 (default 8000), plus the search fields above (no `on_empty` — empty search still returns `pages=[]`).

URL pick: organics with a URL, sorted so `also_from` (cross-confirmed) come first, then first-seen unique URLs up to `fetch_top`. Fetches run in parallel via `asyncio.gather`. Response: `query`, `hits`, `providers_used`, `pages[{url, title, excerpt, ok, is_pdf, tier, error}]`.

GET /search?q=&num=&engine=&api_key=

Drop-in `SerpAPI` shape for scrapers. `engine` and `api_key` are accepted and ignored for search logic (auth still uses `Bearer` or `api_key`). `num` 1..20, default 10.

Pins `SerpAPI` when that provider is configured, so `answer_box` / `knowledge_graph` / `related_questions` survive. If `SerpAPI` is not configured, `providers=None` and the default mix runs. `POST /v1/search` and the chat tools always mix; only this GET pins.

hits_to_serpapi_shape: organics → organic_results[{{title, snippet, link, position}}]. Raw SerpAPI extras used when present; else reconstructed from Hit kinds. Related titles strip the Related: prefix.

6. Mode 2 – chat, loop, tools, DSML

Entry – engine/api/chat.py

Both /chat/completions and /v1/chat/completions call `_handle_chat`.

1. Parse JSON or 400.
2. Lowercase-copy incoming headers (plugin header + X-Priority).
3. `_should_run_loop`: False if Layer 2 is OFF or `phoenician_tools == []`.
4. If stream and the loop would run → 400 (see status table).
5. If stream: `_proxy_stream` — build request to `completions_url()` with `_outbound_chat_body` (always strip `phoenician_*`; strip engine tools if plugin off), forward bytes, media type from upstream. Upstream ≥ 400 → 502. Connect error → 502 with the “Search still works...” / “plugin disabled...” sentence.
6. If not stream: `run_agent_loop`. Map `RequiredSearchFailed` → 424. Upstream `HTTPStatusError` / `RequestError` → 502.
7. Wrap the last upstream JSON and add extras only when they exist: `phoenician_plugin` (always on a successful loop return), `phoenician_tool_trace` + `phoenician_usage_total` if any tools ran, `phoenician_sources`, `phoenician_citations_block`, `phoenician_search_failed`.

Metrics: `ACTIVE_REQUESTS` ± 1 around the try; latency observed on chat; `REQUESTS_TOTAL` status success / error / 424.

Loop – engine/agent/loop.py

`run_agent_loop(client, body, headers):`

1. `_strip_extensions`: pop `phoenician_tools` and `phoenician_web`. None tools → all three. Filter to names in `TOOL_SCHEMAS`.
2. If Layer 1 off, or policy OFF, or managed list empty: optionally `strip_engine_tools` (Layer 1 off only), one `post_completion`, return. No guidance, no tools from us. intelligence on the result is off.
3. Parse `domain_mode`, optional per-request `fetch_mode`, optional `providers` (must be a list or it is ignored).
4. `ToolBudget.from_web_options(web)` — explicit 0 is honored for search/fetch/chars; `max_rounds` is at least 1.
5. Build `ToolExecutor`. `_inject_tools` appends missing schemas; `tool_choice` defaults to auto.
6. `inject_system_guidance` prepends or merges `WEB_SYSTEM_MESSAGE`.
7. Force stream: False. Default model = `UPSTREAM_MODEL`.
8. First `post_completion`.
9. While `rounds < budget.max_rounds`:
 - Take `choices[0]`. Missing choices → `[{}]` then empty message — treated as no tool calls (loop exits, unless required re-ask).
 - `_message_tool_calls`: use structured `tool_calls`, else `recover_tool_calls(content)`. On recovery, write `tool_calls` onto the message and set content to None so markup is not treated as the answer.
 - No calls + policy required + not yet re-asked + not `executor.search_succeeded`: append the assistant message, append user `FORCE_SEARCH_NUDGE` (“You must call the `web_search` tool before answering. Search now.”), set `tool_choice` to force `web_search`, `POST` again, continue (does not increment rounds).
 - No calls otherwise → break.

- Clear `tool_choice` back to `auto` (so a forced re-ask does not stick).
- Append the assistant message. Run every call in parallel (`asyncio.gather`). Unknown or unmanaged name → `ERROR: tool '...' is not available`. Do not present unverified facts as sourced.
- Append role: `tool` with `tool_call_id`. Record trace `{tool, arguments}`.
- `budget.trim_tool_messages`. POST again. `rounds += 1`. Increment `T00L_R0UND5`.

0. After the loop: if policy required and still no `search_succeeded` → raise `RequiredSearchFailed` (reason depends on `search_failed`).

Arguments that are not JSON objects become `{}` (`_parse_args`); the tool then fail-closes on empty query / bad URL rather than 500.

System prompt – `engine/agent/prompts.py`

`WEB_SYSTEM_MESSAGE` tells the model: decide whether this question needs the live web; search for current / company / filing facts; skip greetings, pure math, uncited general knowledge; use the three tools; prefer official filings; never invent URLs; treat `ERROR:` as hard failure; start the answer with no preamble; no trailing Sources section. If the caller already sent a system message, guidance is prepended (unless already present). Otherwise a new system message is inserted at the front.

Upstream HTTP – `engine/agent/upstream.py`

`completions_url`: if the base already ends with `/chat/completions`, use it; if it ends with `/v1`, append `/chat/completions`; else append `/v1/chat/completions`.

`auth_headers`: drop `host`, `content-length`, `authorization`, `connection` from the incoming request. Set `Content-Type` `JSON`. If `UPSTREAM_API_KEY` is set, send that `Bearer` (client `Authorization` is never forwarded). `X-Priority` survives because it is not in the drop set — `ai-router` reads it.

`post_completion` raises on non-2xx. Usage helpers sum `prompt/completion/total` across rounds; missing usage counts as zeros.

Citations – `engine/agent/citations.py`

From `executor.sources` (unique URLs recorded on successful search hits and successful fetches). Two blocks concatenated: numbered `[SOURCE: Web Citation N]` + `[PUBLIC_URL:]`, then titled `[SOURCE: {title} (Date: {date})]`. Entries without a URL are skipped. Empty → `""`.

Tool schemas – `engine/tools/schemas.py`

Name	Required	Optional
<code>web_search</code>	<code>query</code>	<code>num_results</code> , <code>recency_days</code>
<code>fetch_url</code>	<code>url</code>	<code>max_chars</code>
<code>search_and_read</code>	<code>query</code>	<code>top_n</code>

Descriptions tell the model the mix is blended, sources stay visible, and `search_and_read` saves a round. `DEFAULT_T00LS` = all three.

Dispatch – `engine/tools/execute.py`

`_opt_int`: `None/""` → default; `non-int` → default; then clamp `lo/hi`. Used so garbage arguments become an `ERROR` string, not a 500.

`web_search`: empty query → fail-closed search. Budget check before the call; `note_search` after allow.
`num_results` 1..20 (default 8). `SearchUnavailable` or any exception → `search_failed=True` and the ERROR string. Success → `search_succeeded=True`, record hits, `format_hits_for_model`.

`fetch`: budget; URL must be absolute `http(s)` with a host; then `fetch_url`. Failed pages still format as `[PUBLIC_URL: ...]` plus the fail-closed `fetch` line. Successful pages record a source with `source=fetch:{tier}`.

`search_and_read`: empty query → error. `top_n` 1..8 (default 3). Calls `web_search`; if that returned ERROR:, stop. Else take organics, confirmed first, unique URLs, fetch each, concatenate under `--- fetched pages ---`.

Unknown tool name → the same ERROR string as the loop's unmanaged-tool path.

DSML fallback – `engine/tools/dsml_fallback.py`

Used when `tool_calls` is empty. Only recovers names in `{web_search, fetch_url, search_and_read}`. Dedup by (name, arguments JSON).

Three markup families (fullwidth | or ASCII |):

1. Closed DSML invoke: `<|DSML| invoke name="web_search">...<|DSML| parameter name="query">...</...parameter>...</...invoke>`. Loose parameter regex if the closer is missing. If no parameters, the stripped inner text becomes query. If `web_search` has params but no query key, the first value is used as query.
2. Open invoke without a closer: slice to the next invoke or end of string; same parameter parse; require non-empty args.
3. Tokenizer blocks: `<|tool_calls_begin|>...<|tool_calls_end|>` containing `tool_call_begin/end`. Name from `<|tool_sep|>` or a line that matches a known tool. Arguments from JSON, a fenced ````json` blob, or the first `{...}`. `web_search` without a query key is skipped in this family.

Recovered calls get ids `dsml_ + 12 hex chars`, type function, arguments as a JSON string.

7. Search mix – providers, merge, query

Package: engine/search/. Public exports: Hit, SearchOutcome, run_search.

Hit contract – base.py

Fields: title, url, snippet, source, kind (organic | answer_box | knowledge_graph | related_question), position, date, query, also_from.

ALL_PROVIDERS = serpapi, brave, tavily, searxng, google_cse.

DEFAULT_FANOUT (blend order) = serpapi, tavily, brave, searxng. CSE is pin-only.

PROVIDER_PRIORITY (lower wins the source label on merge): serpapi 0, tavily 1, brave 2, google_cse 3, searxng 4.

canonical_url: scheme (default https) + host without www + path without trailing slash. Query string dropped. This is the dedup key.

snippet_key: first 120 chars of snippet, or the canonical URL if snippet is empty.

SearchOutcome.failed is True only when there are errors and zero providers_used (empty hits after a live provider are OK).

HTTP to providers – httputil.py

request_json: attempts = 1 + SEARCH_PROVIDER_RETRIES (default 1 → two tries). Retry on 429/500/502/503 and on timeout / network / remote-protocol. Sleep $0.35 \times \text{attempt}$. Then raise_for_status and .json(). Timeout default SEARCH_PROVIDER_TIMEOUT (15s). CSE uses max(that, 20).

Resolve which providers run – merge.resolve_providers

- No request pin → every name in DEFAULT_FANOUT that is enabled().
- Pin list: keep known names that are configured. Unknown names are logged. If the caller named only unknown providers → SearchUnavailable. If they named known providers but none are configured → SearchUnavailable listing what is configured.
- Zero configured and no pin → SearchUnavailable (“set SERPAPI_KEY, BRAVE_API_KEY, TAVILY_API_KEY, or SEARXNG_URL”).

Fan-out – run_search

All selected providers run in parallel (asyncio.gather(..., return_exceptions=True)). One failure is appended to outcome.errors as {name}: failed (...) and skipped. If none succeed → SearchUnavailable. SerpAPI extras (answer_box, knowledge_graph, related_questions, raw payload) are copied onto the outcome when that provider ran.

Then: dedup_hits → apply_domain_mode → organics blend_organics(..., num_results) + specials appended unchanged. on_empty=error with no hits → unavailable.

Same-URL merge – _merge_organic / collapse_organics

Primary = lower PROVIDER_PRIORITY (SerpAPI wins the source label). Snippet replaced only if the other snippet is ≥ 40 characters longer. also_from collects the other source(s). Title/URL/date/position fall back to the other if the primary is empty. Non-organics are not collapsed here.

Dedup – dedup_hits

After collapse: drop an organic if its canonical URL or snippet key was already kept. Specials dedup on `kind:url_or_snippet`.

Blend – blend organics

Split into confirmed (also from non-empty) vs unique, each grouped by source. Each group sorted by position. Round-robin in DEFAULT_FANOUT order (then any leftover source names). Fill confirmed first up to `num_results`, then unique. `num_results` is at least 1.

Domain filter – apply_domain_mode

Uses `host_denied` from `fetch/policy.py` (same lists as `fetch`). Denied hits are dropped, not errored.

Text for the model – format_hits_for_model

Header `Web search results for: {query}`. Provider warnings listed if any. Groups labeled `[source:kind]`. Each hit: title, optional date, `[also a, b]`, snippet, URL. Zero hits: `No results`. plus partial errors if present.

SerpAPI – serpapi.py

Enabled when `SERPAPI_KEY` is set. GET `https://serpapi.com/search` with `engine=google`, `hl=en`, `gl=us`, `num`, optional `tbs` from `serpapi_tbs_for_recency` (≤ 1 day `qdr:d`, ≤ 7 `qdr:w`, ≤ 31 `qdr:m`, else `qdr:y`). Payload JSON error field → raise (counts as provider failure).

Pace: process-wide lock; wait until `SERPAPI_PACE_SECONDS` (0.4) since the last call. Tests set this to 0.

Parse: organics (dedup inside this payload by `url / snippet[:120]`); `answer_box` as kind `answer_box`; `knowledge_graph` title+description+attributes; first 2 `news_results` as organics at position 50+; first 2 `related_questions` as kind `related_question` titled `Related: {q[:60]}`.

Returns (`hits`, `extras`) — the only provider with that signature. `_call_provider` special-cases it.

Tavily – tavily.py

POST `https://api.tavily.com/search`. Query trimmed to 400 chars; empty → raise. Depth from `TAVILY_SEARCH_DEPTH` if it is one of `ultra-fast` / `fast` / `basic` / `advanced`, else `advanced`. `include_answer=false` (the Brain synthesizes). `chunks_per_source` 5 for `advanced/fast`, else 3. Recency → `time_range` day/week/month/year. JSON error/detail → raise. Parser still accepts an answer string as an `answer_box` hit if the API sends one. Organics use `content` or `raw_content`, `date` = `published_date`.

Brave – brave.py

GET `https://api.search.brave.com/res/v1/web/search` with `X-Subscription-Token`. `count` ≤ 20 , `extra_snippets=true`, `text_decorations=false`, `safesearch=off`. Freshness: `pd` / `pw` / `pm` / `py`. Error field is fatal only when there are no web results. Snippet = `description` + `extra_snippets`. First 2 news results appended as organics (position 40+) if the URL was not already seen.

SearXNG – searxng.py

Enabled when `SEARXNG_URL` is set. GET `{url}/search` `format=json`, `categories=general`, `language=en`. Recency → `time_range` day/week/month/year. Hits from `results[]`.title / `url` / `content|snippet`.

Google CSE — `google_cse.py`

Enabled only when both `GOOGLE_API_KEY` and `GOOGLE_SEARCH_ENGINE_ID` are set. GET `customsearch/v1`, `num` ≤ 10 . Not in the default fan-out; only runs if the caller pins `google_cse`. Recency is ignored (API has no mapping here).

Query helpers — `query.py`

Not called automatically by `run_search`. Callers (or future pipelines) import them.

- `clean_company` — strip PLC/plc/Inc/Limited/Ltd/Group/Corp/Corporation/S.A./SA/ASA...
- `host_from_url / site_query` — `site:{host} {rest}`
- `filetype_pdf` — append `filetype:pdf` unless already present
- `quarter_num` — first digit 1–4 in the quarter string
- `build_simple_queries(ticker, company, quarter, year, ir_domain?)` — up to 15 strings: quoted `company/ticker + 1Q25 / Q1 2025 / results / earnings release`, optional `site:IR` and `q4cdn.com / mziq.com` PDF queries. Deduped. Judgment stays with the caller; this only builds strings.
- Recency mappers used by SerpAPI and Brave (see above).

8. Fetch ladder – policy, tiers, PDF, SEC, JS

Entry: `fetch_url()` in `engine/fetch/ladder.py`. Public package export: `Page`, `fetch_url`.

Page

`url`, `final_url`, `title`, `text`, `content_type`, `is_pdf`, `tier`, `ok`, `error`. `to_dict()` is `dataclasses.asdict`.

Gate + cache

1. Empty URL → failed page “empty url”.
2. `should_skip_fetch` (`facebook.com` / `twitter.com` / `instagram.com`, including subdomains) → “blocked social domain”.
3. `host_denied(url, domain_mode)` → that reason string.
4. Cache hit on the requested URL (module `_PROCESS_CACHE` unless a cache is passed). Metric tier=cache.
5. Otherwise `_fetch_uncached`; unexpected exceptions become a failed page; result stored with ok-based TTL.

Cache – `cache.py`

Thread-safe dict, monotonic deadlines. Success TTL 1800s, failure 120s, max 256 keys. set evicts expired keys, then the soonest-to-expire if still full. get / has drop expired. Hits/misses counters. Tests clear this in the autouse fixture.

Policy – `policy.py`

`deny_list_for`:

- `general_research` and `earnings_doc`: `SOCIAL_SPAM_DOMAINS` only — `facebook.com`, `instagram.com`, `twitter.com`, `x.com`, `tiktok.com`.
- `ir_discovery`: union (deduped) of `IR_DISCOVERY_BLOCKLIST` + `AGGREGATOR_DOMAINS` + `SOCIAL_SPAM`. IR list includes `yahoo`, `bloomberg`, `sec.gov` / `edgar.sec.gov`, `nasdaq`, `nyse`, `asx`, `marketwatch`, `annualreports`, `morningstar`, `reuters`, `investing`, `fool`, `seekingalpha`. Aggregators include `wikipedia`, `linkedin`, `reuters`, `google`, `youtube`, `marketscreener`, `ft`, `wsj`, `tradingview`, `cnbc`, `crunchbase`, `PR wires`, `seekingalpha`, `fool`, `barrons`, `forbes`, `tipranks`, `finance.yahoo`, `msn`, and the rest listed in the file. SEC is blocked in IR discovery; `earnings_doc` allows it.

Host match: exact or subdomain (`host.endswith("." + domain)`). `www`. stripped first.

`ALWAYS_SKIP_FETCH` (all modes, before deny lists): `facebook.com`, `twitter.com`, `instagram.com`.

WAF: status in {401, 403, 406, 409, 429, 503} or first 5 kB of body containing any of: “please verify you are a human”, “attention required! | cloudflare”, “sucuri website firewall”, “pardon the interruption”, “are you a robot”, “403 forbidden”.

Age-gate cookies attached on every httpx GET: `age_verified` / `over18` / `legal_age` / `is_over_18` = true.

Headers: Chrome 122 UA, HTML Accept, en-US, Accept-Encoding from `supported_accept_encoding()` — `gzip`, `deflate`, and `br` only if `brotli` or `brotlicffi` import. Advertising `br` without a decoder used to leave compressed bytes treated as page text.

`is_ir_url`: “ir.” or “investor” in the URL. `is_sec_wrapper_url`: `sec.gov`, or `cdn.yahoofinance.com` with `sec-filings`.

Ladder – `_fetch_uncached`

SEC URLs use `sec_headers()` (descriptive EDGAR UA). PDF-looking URLs use PDF Accept/Referer headers.

1. **Direct PDF** (URL looks like PDF, not sec.gov): `_download_pdf_resilient` — `httpx`; if 200 and magic/ctype look like PDF, parse. If blocked status or HTTP error, `curl_cffi` (unless `httpx_only`), then Playwright PDF GET (if full). Returns that Page or None (fall through).
2. **IR HTML + FETCH_MODE=full**: Playwright first. If rendered text $\geq$ `THIN_HTML_CHAR_THRESHOLD` (400), return tier=playwright. Otherwise continue.
3. **Tier 1 httpx**. Status $\geq$ 400 that is not a bot-block status → failed Page immediately (no escalate). Bot-block status or challenge HTML or undecodable body (null bytes or $<85\%$ printable in first 500 chars — typically raw brotli) → escalate. Success:
 - SEC wrapper HTML → `handle_sec`; if text, return tier=sec.
 - Else extract. If HTML is thin and mode is full, try Playwright and keep it if longer; else return the thin `httpx` page.
4. **Tier 2 curl_cffi** (not `httpx_only`): impersonate Chrome. Status 200 + body → Page tier=`curl_cffi`.
5. **Tier 3 Playwright** (full only): PDF path uses `playwright_pdf`; HTML uses `playwright_fetch`.
6. Else failed “all fetch tiers failed (last_status=N)”.

`looks_like_pdf`: body starts with %PDF, or Content-Type contains pdf and the body is not HTML, or URL says PDF and magic matches.

Caps: `max_chars` if passed, else 8000 HTML / 25000 PDF. Download hard cap for PDF parse is `MAX_PDF_BYTES` (25 MB) — larger PDFs skip parsers and return None from `pdf_to_text`, which becomes empty text then the placeholder in the parser.

Playwright HTML – `playwright_fetch + js_interact.py`

Headless Chromium, UA + 1280×800, age cookies, goto wait_until=domcontentloaded (timeout `HEADLESS_RENDER_TIMEOUT_MS`, 15s). On goto failure, try the origin/referer. Then `dismiss_age_gate` then `interact_page`.

`interact_page`:

1. Wait networkidle 4s (ignore timeout). Wait spinners hidden 1.5s (selectors: `.loading`, `.spinner`, `.loader`, `.is-loading`, `[class*='loading|spinner|loader']`, `[aria-busy=true]`).
2. If URL fragment, scroll that id/name into view.
3. Scroll to bottom and back. Snapshot HTML + same-origin iframes (skip about:/data:, skip frames <200 chars). Iframe HTML has links absolutized. Snapshots hashed; duplicates dropped. Joined later with `<!-- interaction-snapshot --> / <!-- iframe-snapshot -->`.
4. Click expanders by accessible name (max 8 clicks, 2 per pattern): load/show/view/see more, view/show/see all, more results, more, read more, expand, older, archive, and Japanese もっと見る / 一覧 / すべて / さらに表示 / 過去 / もっと読む. Roles button and link. 400ms pause + snapshot after each click.
5. Click tabs: CSS list (role=tab, aria-expanded=false, Bootstrap/nav-tabs, accordion titles, data-tab / data-toggle=tab) up to 12, then label regexes (financial results, quarterly, interim, annual report, presentations, earnings, Polish raporty..., Japanese 決算 / 四半期 / 有価証券報告書 / 決算短信 / 適時開示 / IR資料 / 財務 / 報告書) up to 16 total tab clicks.
6. Harvest JS: every a[href], data-url / data-href / data-pdf / data-file / data-src / data-download / data-document / data-link / data-target-url / data-attachment / data-doc, any data-* containing http or .pdf,

and http URLs inside onclick. Keep PDFs and hosts matching xj-storage.jp, /xcontents/, irpocket, q4cdn, mziq, cloudfront, /documents/, /attachment, filedownload. Append a tiny HTML blob of those anchors labeled harvested-dynamic-links.

Age-gate clicks (first match wins): I am over 21/18, legal age, yes, enter, verify, confirm, I agree, agree, continue — button or link.

PDF — pdf.py

Singleton pdf_parser. Order, first success wins:

1. pdfplumber — extract_text per page, join with newlines.
2. PyMuPDF (fitz) — get_text per page.
3. pypdf PdfReader(strict=False) — per-page extract, skip pages that throw.
4. poppler pdftotext -layout via temp files (30s). Missing binary → skip.
5. latin-1 decode + regex of readable runs; keep if >100 chars (raw_regex).

All fail → literal This document is a PDF report available at: {url} (the real URL, not invented text). Whitespace collapsed. Oversize body → None (then the placeholder path). Logs which parser won.

SEC — sec.py

UA: SEC_EDGAR_USER_AGENT or a default EarningsTracker-compatible string. extract_sec_exhibit_urls regexes EX-99.1 / EX-99.2 / EX-99 / exhibitN in hrefs and link text. Resolve //, /, relative. Sort: PDF first, then EX-99.1 before other exhibits.

Yahoo Finance wrappers: same-host .htm/.pdf under the same path prefix, ranked by link text (earnings release → press release → results → presentations → 6-k → exhibit).

handle_sec: try first 5 exhibit URLs. PDF if ctype/url says so and text >500 chars; else SEC HTML extract >500. Then wrapper body extract; then <DOCUMENT><TYPE>EX-99.1|6-K<TEXT> embedded. HTML extract prefers #documentContent / .body / #content / .content / <body> if the stripped text is >200 chars; strips SEC boilerplate (Page N of M, CIK/SIC, sec.gov). Returns (text, is_pdf, resolved_url) or (None, False, original).

Extract — extract.py

extract_html → (text, title, canonical). Trafilatura markdown with links+tables if it yields substance (timeout 0 = no extra watchdog). Metadata for title/url. Fallback: html_to_text — drop script/style/svg/head, turn block tags into newlines, strip tags, unescape nbsp/amp/lt/gt/quot/euro/pound/yen, drop “Cookie Policy / Accept Cookies / Skip to content” chrome. Title from <title> if needed; canonical from rel=canonical.

URL helpers — urlutil.py

resolve_href / absolutize_html_links. is_pdf_url: path ends with .pdf or filetype=pdf. extract_all_links skips mailto/javascript/tel, cookie/privacy/terms, and social hrefs. looks_like_html_url: .htm/.html or “htm” in the path (used to decide whether a SEC URL should hunt exhibits).

9. Supporting modules

Budget — engine/budget.py (re-exported as engine/agent/budget.py)

from_web_options: keys max_search_uses, max_fetches, max_rounds, web_context_budget_chars.

Missing/None/"" → settings. Garbage int → default. Explicit 0 is kept (search/fetch then immediately fail-closed). max_rounds = max(1, n). Chars/search/fetch floored at 0.

trim_tool_messages: if char budget ≤ 0, no-op (unlimited). Else while the sum of role=tool contents exceeds the cap, compact the oldest tool message that is not already a stub and is ≥ 80 chars. Stub:

[compacted: {dropped} chars dropped; URLs retained: {urls}]. URLs collected from [PUBLIC_URL: ...] lines and bare http(s) tokens.

Errors — engine/errors.py

- SearchUnavailable.message — Mode 1 → 502; tools → that string.
- RequiredSearchFailed.reason — Mode 2 → 424.
- StreamToolsRejected — defined; chat uses HTTPException 400 directly.
- Helpers: fail_closed_search / fetch / budget all end with Do not present unverified facts as sourced.

Connect — engine/connect.py

connection_info() for GET / and GET /v1: OpenAI base http://127.0.0.1:{port}/v1, DeepSeek-style base without /v1, model, api_key or the literal local, snippets for OpenAI Python, env exports, Cursor, Continue, LiteLLM. Notes list the two plugin layers and that Mode 1 does not need a Brain. print_banner is what python -m engine shows.

Setup — engine/setup.py

If .env is missing, copy .env.example. Print whether SERPAPI / Tavily / Brave keys exist (never the values). Print banner + start/toggle/check commands. --check exits 1 if no search key.

10. File map of the repo

engine/	the service (the only package)
__init__.py	__version__ = "0.1.0"
__main__.py	python -m engine → run setup check
main.py	FastAPI app, lifespan, CORS, router mount
config.py	env → Settings + enums + VLLM_REQUIRED_FLAGS
runtime.py	http_client, fetch_cache, plugin_enabled, capabilities
plugin.py	Layer 1 / Layer 2
auth.py	Bearer / ?api_key=
errors.py	typed failures + ERROR: strings
budget.py	per-request caps + tool-message compaction
metrics.py	Prometheus names
connect.py	client snippets
setup.py	create .env from example, print connect info
search/	providers + merge + query hygiene
__init__.py	Hit, SearchOutcome, run_search
base.py	Hit / SearchOutcome / canonical_url / fan-out order
httputil.py	JSON GET/POST with one cheap retry
merge.py	fan-out, collapse, blend, domain filter, format
query.py	company clean, site:pdf queries, recency tbs
serpapi.py	Google SERP + extras + 0.4s pace
tavily.py	long excerpts, include_answer=false
brave.py	independent index + extra_snippets + news
searxng.py	self-hosted JSON
google_cse.py	pin-only Custom Search
fetch/	download + extract
__init__.py	Page, fetch_url
ladder.py	httpx → curl_cffi → Playwright → PDF/SEC
policy.py	deny lists, WAF markers, headers, IR/SEC predicates
cache.py	TTL cache, shorter TTL on failure
extract.py	Trafilatura then html_to_text
pdf.py	pdfplumber → fitz → pypdf → pdftotext → regex
sec.py	EX-99 discovery, Yahoo wrapper, EDGAR UA
js_interact.py	expanders, tabs, age-gate, iframes, data-* harvest
urlutil.py	href resolve, PDF detect, title/canonical, links
tools/	
__init__.py	KNOWN_TOOLS, TOOL_SCHEMAS
schemas.py	OpenAI function JSON for the three tools
execute.py	dispatch, budget, fail-closed strings, sources
dsm_l_fallback.py	recover tool_calls from DeepSeek markup
agent/	
__init__.py	run_agent_loop
loop.py	tool loop, canary, strip helpers
budget.py	re-export of engine.budget.ToolBudget
prompts.py	WEB_SYSTEM_MESSAGE + FORCE_SEARCH_NUDGE
citations.py	[SOURCE]/[PUBLIC_URL] blocks
upstream.py	completions URL, auth headers, usage sum
api/	
__init__.py	package marker
health.py	/, /health, /metrics, /capabilities
ui.py	GET /ui
plugin.py	GET/POST /v1/plugin
openai_compat.py	/v1, /v1/models
chat.py	completions + stream proxy
primitives.py	/v1/search /fetch /research
compat.py	GET /search SerpAPI shape
tests/	pytest (no live keys)
scripts/	check_search.py, smoke.py, install.sh
docs/	this file, render_pdf.py, PDF
Dockerfile	python:3.12-slim + poppler + Chromium
docker-compose.yml	engine :8090; optional searxng profile

Not product code: .cursor/research/ (build notes), org-repos/ (gitignored clones). This paper does not document those.

11. Tests – what each file locks

tests/conftest.py autouse fixture: blank every search key, upstream http://upstream.test, fetch_mode=no_browser, canary off, plugin on, policy auto, pace 0, retries 0. Restores settings and runtime.plugin_enabled. Clears _PROCESS_CACHE. client fixture is a FastAPI TestClient of engine.main:app (runs lifespan).

No test talks to a live provider or a live Brain. ~71 tests.

File	What it locks
test_plugin.py	Welcome/health expose plugin. Toggle blocks Mode 1 with 503. Header off without flipping global. Global off wins over body/header on. Body enabled: false is pass-through. Intelligence auto when on. Status explains layers. /ui HTML. Plugin off does not start the loop. Policy off skips loop including stream. Strip phoenician_* and engine tools. Stream outbound body stripped when plugin off.
test_loop.py	Tool round then final prose. DSML in content recovered. policy=required: re-ask then 424; re-ask then search succeeds. Policy off / plugin disabled are pass-through. Schema is OpenAI function shape.
test_fail_closed.py	No providers never says “own knowledge”. Over-budget is ERROR string. Helper wording. Empty query and bad fetch URL are ERROR, not 500.
test_budget.py	Search/fetch caps. Oldest tool output compacted first. from_web_options honors explicit 0; ignores garbage.
test_api.py	Health, welcome curls, /v1/models, capabilities lists vLLM flags. stream+tools → 400. Auth 401 when key set. Empty search query → 400. GET /search accepts api_key= .
test_compat.py	Hits → organic_results / answer_box / knowledge_graph / related_questions shape.
test_merge.py	Collapse keeps Google source + longer Tavily snippet. Blend interleaves and promotes confirmed. Dedup by canonical URL and snippet. ir_discovery drops SEC + aggregators. earnings_doc allows SEC. general_research drops social only. Pin of unconfigured provider is a clear error. SerpAPI JSON error field is a provider failure. All fail raises; one fail is skipped.
test_providers.py	Tavily parse of answer + results. Brave parse of web, news, extra_snippets.
test_serpapi_parse.py	Organic + answer_box + KG + two related. extras keep raw blocks.
test_query.py	Legal-suffix strip. site: / filetype builders. build_simple_queries includes PDF and site:.
test_ladder.py	Accept-Encoding only advertises decodable. Binary rejected. PDF magic vs HTML disguise. 403 escalates to curl_cffi. httpx_only does not. Thin HTML escalates to Playwright. PDF bytes go through the parser. SEC wrapper follows EX-99.1. Exhibit URL sort priority. Social skipped.
test_extract.py	html_to_text strips chrome and unescapes. Fallback when Trafilatura empty. max_chars respected.
test_dsml.py	Closed invoke, open invoke without closer, DeepSeek tool__calls markup. Empty/prose → [].

12. Scripts and Docker

scripts/check_search.py

Live Mode 1. Query “Phoenician Capital”. Default: try the engine at `ENGINE_URL` (default `http://127.0.0.1:8090`); if connect fails, fall back to calling `run_search` in-process. `--engine` fails if `/health` is down. `--direct` never uses the server.

Via engine: GET `/` (print `search_ready`, providers, plugin). If plugin off → FAIL with the `/ui` message. If no providers → FAIL. Then POST `/v1/search` (require hits), GET `/search` (print organic count), POST `/v1/fetch` example.com with `httpx_only` (require ok). Bearer from `ENGINE_API_KEY` if set. Never prints keys.

scripts/smoke.py

Live Mode 2 against a running engine + Brain. GET `/capabilities` (warn if canary failed). POST chat with `policy: required` and a dated Apple-revenue question — require `web_search` or `search_and_read` in the trace and an http URL in sources; 424 is FAIL. Then POST “What is 2+2?” with `policy: off` — FAIL if a tool trace appears. Timeout 180s.

scripts/install.sh

Create `.venv` if missing, `pip install -r requirements-dev.txt, python -m engine setup`. Requires python3.

docs/render_pdf.py

Playwright Chromium opens `method.html` as a file:// URI, writes `docs/Local-LLM-Internet-Engine-Method.pdf` A4 with header “Phoenician Capital · local-engine-internet / How it works · 9 September 2026” and a page number footer. Use `.venv/bin/python docs/render_pdf.py`.

Docker

Dockerfile: `python:3.12-slim`. apt: `poppler-utils, wget, gnupg, ca-certificates`. `pip install requirements.txt`, then `playwright install --with-deps chromium`. Copies only engine/ (not tests/scripts/docs). CMD `uvicorn engine.main:app --host 0.0.0.0 --port 8090`.

`docker-compose.yml`: service engine build `.`, ports 8090:8090, optional `.env`, restart unless-stopped. Service `searxng` image `searxng/searxng:latest` under profile `searxng`, host 8888 → container 8080. Start with `docker compose up --build`. Add SearXNG: `docker compose --profile searxng up` and set `SEARXNG_URL`.

13. Environment

Read once at import from process env + `.env`. Defaults below match `engine/config.py`. `.env.example` sets `FETCH_MODE=no_browser`; the Settings default if unset is `full`.

Variable	Default	Meaning
ENGINE_HOST / ENGINE_PORT	0.0.0.0 / 8090	Bind
ENGINE_API_KEY / SEARCH_API_KEY	empty	Incoming auth; unset = open
PLUGIN_ENABLED	true	Layer 1 at process start
DEFAULT_WEB_POLICY	auto	Layer 2
DEFAULT_DOMAIN_MODE	general_research	Search/fetch deny list
UPSTREAM_LLM_BASE_URL	http://127.0.0.1:8080	Mode 2 LLM
UPSTREAM_API_KEY / ROUTER_API_KEY	empty	Bearer to the LLM
UPSTREAM_MODEL	deepseek-v4-flash	Default model field + /v1/models
SERPAPI_KEY / SERPAPI_API_KEY		Google SERP
TAVILY_API_KEY		Long excerpts
BRAVE_API_KEY		Independent index
SEARXNG_URL		Self-hosted search
GOOGLE_API_KEY + GOOGLE_SEARCH_ENGINE_ID		CSE, pin-only
TAVILY_SEARCH_DEPTH	advanced	ultra-fast / fast / basic / advanced
SEARCH_PROVIDER_RETRIES	1	Extra tries after the first
SEARCH_PROVIDER_TIMEOUT	15s	Per provider HTTP
SERPAPI_PACE_SECONDS	0.4	Min gap between SerpAPI calls
FETCH_MODE	full (example: no_browser)	full / no_browser / httpx_only
DOC_FETCH_TIMEOUT	20s	Page GET
HEADLESS_RENDER_TIMEOUT_MS	15000	Playwright goto
THIN_HTML_CHAR_THRESHOLD	400	Escalate thin pages
MAX_CHARS_PER_DOC / PDF	8000 / 25000	Extract caps
MAX_TOTAL_DOC_CHARS	50000	In Settings; not applied in the current ladder/loop
MAX_PDF_BYTES	25 MiB	Skip parse above this
FETCH_CACHE_TTL	1800s	Success cache (lifespan cache object)
FETCH_CACHE_FAILURE_TTL	120s	Negative cache
MAX_TOOL_ROUNDS	6	Loop cap (at least 1)
MAX_SEARCH_USES / MAX_FETCHES	8 / 8	Per request
WEB_CONTEXT_BUDGET_CHARS	150000	Compact oldest tool text over this
DEFAULT_NUM_RESULTS	8	web_search default
DEFAULT_SEARCH_AND_READ_TOP_N	3	search_and_read default
REQUEST_TIMEOUT	300s	Upstream LLM + httpx read
SEC_EDGAR_USER_AGENT	LocalEngineInternet/1.0 ...	EDGAR requires a contact UA
RUN_CANARY_ON_STARTUP	true	Background tool-call probe
CANARY_TIMEOUT_SECONDS	5	Does not block listen
CORS_ALLOWED_ORIGINS	empty	Comma list; empty + no API key → *

14. How to run

```
python3.12 -m venv .venv && source .venv/bin/activate
pip install -r requirements-dev.txt
cp .env.example .env          # set SERPAPI_KEY
python -m engine              # :8090
python scripts/check_search.py
pytest
```

Clients: OpenAI-compatible base `http://127.0.0.1:8090/v1`, key `local` if unset. DeepSeek-style base without `/v1`: `http://127.0.0.1:8090`. Toggle: `/ui`. OpenAPI: `/docs`.

Mode 2 needs a Brain at `UPSTREAM_LLM_BASE_URL` that emits OpenAI `tool_calls` (or DSML the fallback can parse). vLLM DeepSeek Flash flags are in §2.

Docker: `docker compose up --build`. `FETCH_MODE=full` locally needs playwright install chromium (the image already installs it).

PDF of this file: `.venv/bin/python docs/render_pdf.py`.

Optional chat body (stripped before the LLM): `phoenician_web.{enabled, policy, max_search_uses, max_fetches, max_rounds, web_context_budget_chars, fetch_mode, domain_mode, providers}` and `phoenician_tools`. Header `X-Phoenician-Plugin: off` forces Layer 1 off for that request only.